Sparsity Is Not Speed: An Exhaustive CPU-Latency Analysis of Structured Pruning for Hardware-Aware RL

[Author Name]
[Department]
[Organization]
[City, Country]
[e-mail address]

***Abstract*—Reinforcement-learning (RL) pruning methods usually reward sparsity, assuming that fewer parameters mean faster inference. We test this assumption exhaustively on a commodity CPU before training any agent. For CIFAR-10/100 SimpleCNN and ResNet-8, we physically compact all 1,296 policies of a four-unit, six-action structured layer-wise pruning space and measure batch-1, single-thread latency on an Intel Core i7-1260P with a pre-registered protocol (performance-core pinning, five fresh-process sessions, dense anchors). Parameter reduction is a weak predictor of latency gain (Spearman ρ = 0.31–0.55), whereas MAC reduction is a strong one (ρ = 0.89–0.98); in 31–39% of policy pairs, the sparser policy is the slower one. A SimpleCNN policy that removes 51% of the parameters is only 1.03× faster, while one that removes 19% is 2.08× faster. We then measure how much latency an accuracy-constrained search could gain. With a fixed 5-epoch recovery and a pre-registered 1-epoch proxy (accepted for ResNet-8, ρ = 0.98; rejected for SimpleCNN, ρ = 0.84), the fastest ResNet-8 policy predicted to keep 98% of the recovered validation accuracy is only 1.098× faster than the dense model. Hardware-aware pruning rewards should therefore use measured or MAC-based cost, and the latency headroom of a pruning problem should be measured before an RL search is built on it.**

Keywords—structured pruning, CPU inference latency, hardware-aware neural network compression, reinforcement learning, proxy validation, reproducibility

I. Introduction

Pruning removes weights, channels or neurons from a trained network to make inference cheaper [1]–[3]. Layer-wise methods decide how much to remove from each layer, and several of them search this allocation with reinforcement learning (RL) [5] or derive it from layer-adaptive scores [6], [7]. The search is usually driven by a proxy for efficiency, such as the fraction of removed parameters or the number of floating-point operations. These proxies are convenient, but what a deployed system pays for is time.

Unstructured pruning makes the gap between proxy and time explicit: zeroed weights stay inside dense tensors, and dense CPU kernels do not skip them, so latency is unchanged [11], [12]. Structured pruning removes whole filters or neurons, so tensors physically shrink [2], [4]. Even then, the parameter count does not determine latency. Parameters and arithmetic are distributed differently across layers, and every inference also pays a fixed framework overhead. Platform-aware methods therefore measure latency directly or through lookup tables [5], [8]–[10].

This paper asks two questions that should be answered before a latency-aware RL pruning agent is trained. First, on a commodity CPU, how well do parameter sparsity and multiply–accumulate (MAC) count predict the measured latency of structured pruning policies? Second, once the pruned network is retrained under a fixed recovery budget and must keep a fixed share of its accuracy, is enough latency left to optimize? We answer both exhaustively for a finite space of $6^4 = 1{,}296$ layer-wise policies on CIFAR-scale networks. Every protocol and decision rule was committed to version control before the corresponding data were collected. Our contributions are:

- an exhaustive, pre-registered latency study of all 1,296 physically compacted policies of four network and dataset pairs, timed at batch size 1 on a pinned performance core in five fresh-process sessions;
- evidence that parameter reduction is a poor latency proxy (ρ = 0.31–0.55) and MAC reduction a good one (ρ = 0.89–0.98), with 31–39% of policy pairs inverted for parameters;
- a validated proxy design, in which a 1-epoch recovery proxy is accepted for ResNet-8 (ρ = 0.980 against 5-epoch recovery) and rejected for SimpleCNN (ρ = 0.844) by a pre-registered rule;
- a pre-registered feasibility analysis showing that ResNet-8 policies predicted to keep 98% of the recovered accuracy are at most 1.098× faster, so a latency-aware search has little room to act on this platform.

II. Related Work

Pruning granularity. Magnitude pruning of individual weights yields high sparsity [1], but irregular sparsity needs specialized kernels to become faster [12]. Filter pruning ranks whole filters, for example by their L1 norm, and removes them [2]; Liu *et al.* showed that the resulting architecture matters more than the inherited weights [3]. In residual networks, channels joined by an addition must be removed together, and DepGraph formalizes these dependency groups [4].

Layer-wise allocation. AMC trains an RL agent to choose per-layer compression ratios under a resource-constrained or accuracy-guaranteed reward [5]. LAMP derives layer-adaptive sparsity from normalized magnitude scores [6], and ERK sets layer densities from tensor shapes [7]. Proximal policy optimization [13] is a common choice for such agents.

Hardware awareness. NetAdapt adapts a network to a target platform with empirical latency measurements [8]. ProxylessNAS and FBNet use measured or table-based latency inside neural architecture search [9], [10]. Blalock *et al.* criticize pruning results that report only parameter or FLOP

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

reductions [11]. Our study does not propose a search method. It measures, for an entire finite policy space, how much latency the common proxies explain and how much latency remains under an accuracy constraint.

III. Experimental Setup

A. Networks and Data

We use CIFAR-10 and CIFAR-100 [15]. The 50,000 training images are split once into 45,000 for training and 5,000 for validation. The validation images are divided into two stratified halves of 2,500: VAL-RL, used for every accuracy reported here, and VAL-SELECT, which is held back. The official test set is not used in this study. The networks are SimpleCNN (three 3×3 convolutions with 32, 64 and 128 filters, a 256-unit hidden layer and a linear classifier; 620,362 parameters) and a CIFAR ResNet-8 [14] (a 16-channel stem and three basic blocks with 16, 32 and 64 channels; 78,042 parameters). Each pruned model starts from a frozen, hash-verified dense reference trained in an earlier stage of this work. All code uses PyTorch 2.7.1 [16].

B. Structured Action Space

Each network has four pruning units, and each unit takes one of six actions that remove $r \in \{0, 10, 20, 30, 40, 60\}$% of its channels. A unit with C channels keeps

$$k = \max(1, \text{round}((1 - r)C)) \tag{1}$$

channels with the largest L1 norm [2], scored on the dense model so that the result does not depend on the order of the units. For SimpleCNN the units are the output filters of the three convolutions and the hidden neurons of the first fully connected layer. The following layer loses the matching inputs, including the 16 flattened columns of each removed channel, and the classifier is never pruned. For ResNet-8 the first unit is the stage-1 residual width. One channel set is removed from the stem convolution and its batch normalization, from the outputs of the second convolution of block 1, and from the inputs of every layer that reads this residual stream, including the identity and projection shortcuts [4]. These channels are ranked by the summed L1 norm of the two filters that write them. The other three units are the internal channels of the three blocks.

All 1,296 action vectors give distinct physical shapes in both networks. Slimmed models and the equivalent masked models give the same logits (maximum difference 9.5×10^{-6}). Parameter and MAC counts equal closed-form values for all 1,296 policies, and the FLOPs reported by PyTorch's FlopCounter equal twice the MACs.

C. Latency Measurement

All timing runs on a laptop with an Intel Core i7-1260P (four performance and eight efficiency cores), Windows 11 and the PyTorch 2.7.1 CPU build with oneDNN 3.7.1, on AC power. The primary configuration uses batch size 1, one intra-op and one inter-op thread, and a process pinned to one hyper-thread of a performance core before PyTorch is imported. Each model is timed in evaluation and inference mode on a single preallocated float32 input; model construction is never timed. Every shape receives 50 warm-up passes and 300 timed passes, measured with a nanosecond counter.

A latency table covers all 1,296 shapes of one network and is measured in five fresh-process sessions, each in a seeded random order. A block of dense-model passes after every 25 shapes monitors drift, and a session whose anchors drift by more than 10% is repeated. The latency L of a shape is the median over sessions of its per-session median, and its gain is

$$g_L = 1 - L / L_{\text{dense}} \tag{2}$$

where L_{dense} is the unpruned shape of the same table.

D. Recovery and Pre-Registered Decision Rules

Removing channels without retraining is destructive: removing 10% of every unit of ResNet-8 leaves 21% validation accuracy. Every policy is therefore recovered with one fixed protocol, inherited unchanged from earlier experiments of this work: Adam with learning rate 10^{-4}, cosine annealing over the budget, batch size 128, random crop and horizontal flip, no weight decay, and one fixed seed that gives every policy the same minibatch order. We report the retained accuracy

$$q = A / A_{\text{dense}} \tag{3}$$

where A is the VAL-RL accuracy after recovery and A_{dense} is that of the dense model after the same recovery. A full 5-epoch recovery of every policy would take about 130 CPU-hours on this laptop. We therefore use the first recovery epoch as a search proxy (q_1) and the full five epochs (q_5) for every claim. Three rules were committed before their data existed:

- the proxy is accepted for a network only if the Spearman correlation between 1-epoch and 5-epoch accuracy is at least 0.90;
- the proxy threshold τ is the largest q_1 that keeps every policy with $q_5 \geq 0.98$ in a stratified calibration sample of 48 policies and leaves at least 20 policies in the full table, minus a margin of 0.005;
- a latency-aware search proceeds only if at least 20 policies pass τ and at least one of them is at least 1.10× faster than dense.

IV. Results

A. Choice of the Latency Metric

Before building the tables, we timed six SimpleCNN calibration models in three configurations: batch 1 with one thread (A), batch 1 with four threads (B), and batch 128 with four threads (C). A configuration was usable if the latency difference between the dense model and the most slimmed model was at least five times the between-session spread, at least 10% of the dense latency, and positive in every session. The first usable configuration in the order A, C, B became the primary metric. Configuration A gave a signal-to-noise ratio of 12.3, C 9.6 and B 4.3, so batch-1, single-thread latency is the primary metric (Table I). The between-session coefficient of variation of the dense model was 2.1%. Even in this small set, the fc-heavy model removes 51% of the parameters but only 3% of the MACs and is just 1.03× faster. The conv-heavy model removes 63% of the parameters and 65% of the MACs and is 1.47× faster.

TABLE I. LATENCY CALIBRATION ON CIFAR-10 SIMPLECNN

Model	Params ↓	MACs ↓	Median ms	Speedup
Dense	—	—	0.680	1.00×
Light	19.1%	17.5%	0.609	1.12×
Moderate	50.7%	49.5%	0.454	1.50×
Aggressive	83.9%	81.7%	0.261	2.60×
FC-heavy	51.1%	2.9%	0.657	1.03×
Conv-heavy	62.6%	65.2%	0.465	1.47×

Light, moderate and aggressive remove 10%, 30% and 60% of every unit; FC-heavy removes 60% of fc1; conv-heavy removes 60% of conv2 and conv3. Configuration A (batch 1, one thread on a pinned performance core), five sessions. Signal-to-noise: A 12.3, C 9.6, B 4.3.

B. *Exhaustive Latency Tables*

Table II summarizes the four tables. SimpleCNN shapes span 0.30–0.79 ms (up to 2.61× faster than dense), and ResNet-8 shapes span 0.46–0.77 ms (up to 1.67×). Across sessions, the median per-shape coefficient of variation was 1.2–1.6%, and the rank correlation of latency gains between any two sessions was at least 0.96. The absolute dense latency differed by 16% between the calibration day and the table measurements, while relative gains were stable. We therefore interpret only gains relative to a dense model measured in the same sessions.

Parameter reduction is a weak predictor of latency gain (Fig. 1a): Spearman ρ is 0.31–0.32 for SimpleCNN and 0.54–0.55 for ResNet-8. MAC reduction predicts it well (Fig. 1b; ρ = 0.89 and 0.98). Across all pairs of policies with different parameter reductions, the sparser policy is the slower one in 39% of SimpleCNN pairs and 31% of ResNet-8 pairs. For MAC reduction, the inversion rate drops to 14% and 5–6%.

The extremes are instructive. The SimpleCNN policy [5, 5, 1, 0] removes only 18.7% of the parameters but 69.3% of the MACs and runs 2.08× faster. Removing 60% of the fully connected layer, which holds most parameters but few MACs, removes 51% of the parameters for a 1.03× speed-up. A linear fit of latency on MACs gives an intercept of 0.26 ms for SimpleCNN ($R^2 = 0.81$) and 0.33 ms for ResNet-8 ($R^2 = 0.97$). This fixed per-inference cost is not reduced by any channel removal, and it caps the attainable speed-up.

TABLE II. EXHAUSTIVE LATENCY TABLES (1,296 POLICIES EACH)

Network	Max speedup	ρ_P	ρ_M	Sparser, slower
SimpleCNN, C10	2.61×	0.32	0.89	39.0%
SimpleCNN, C100	2.59×	0.31	0.89	39.4%
ResNet-8, C10	1.67×	0.55	0.98	30.6%
ResNet-8, C100	1.72×	0.54	0.98	30.8%

ρ_P, ρ_M: Spearman correlation of parameter and MAC reduction with latency gain. Last column: share of policy pairs in which the policy with the larger parameter reduction has the higher latency.

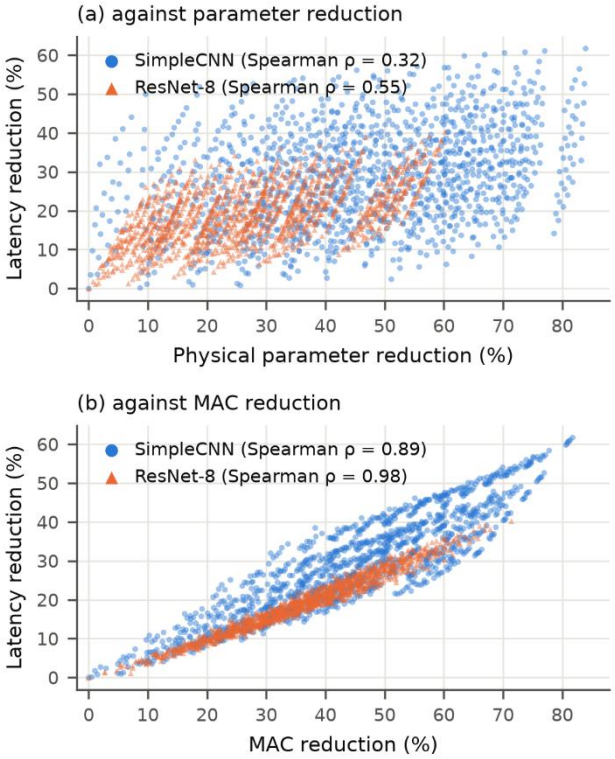


Fig. 1. Latency reduction of all 1,296 physically compacted policies of CIFAR-10 SimpleCNN and ResNet-8 against (a) physical parameter reduction and (b) MAC reduction (batch 1, one thread, i7-1260P).

C. *Recovery and Proxy Validity*

Five epochs of recovery restore most of the accuracy lost to channel removal. ResNet-8 with 10% removed from every unit improves from 20.96% to 76.84% VAL-RL accuracy, and the conv-heavy SimpleCNN improves from 25.84% to 73.36%. The 1-epoch proxy was validated on 156 SimpleCNN policies (126 lightly pruned pilot policies and 30 random ones) and on 31 ResNet-8 policies (the dense policy and 30 random ones), each recovered for one and for five epochs (Fig. 2, Table III).

The proxy underestimates the 5-epoch accuracy in every pair, by 1.29 percentage points on average for SimpleCNN and 5.68 points for ResNet-8. For ResNet-8 it preserves the ranking (ρ = 0.980) and was accepted. For SimpleCNN the overall ρ is 0.844, below the pre-registered 0.90, and the proxy was rejected. Among the lightly pruned pilot policies, whose 5-epoch accuracies lie within a few tenths of a point of each other, the 1-epoch ranking is unreliable (ρ = 0.77), although it is accurate across the random policies (ρ = 0.97). We therefore did not build a SimpleCNN accuracy table, and SimpleCNN contributes to this paper only through its latency evidence.

TABLE III. VALIDATION OF THE 1-EPOCH RECOVERY PROXY

Metric	SimpleCNN	ResNet-8
Policy pairs	156	31
Spearman ρ (A_1, A_5)	0.844	0.980
Pearson r	0.973	0.990
Mean bias $A_1 - A_5$ (pp)	−1.29	−5.68
Top-10 overlap	3	10
Decision (ρ ≥ 0.90)	Rejected	Accepted

A_1, A_5: VAL-RL accuracy after 1 and 5 recovery epochs (CIFAR-10).

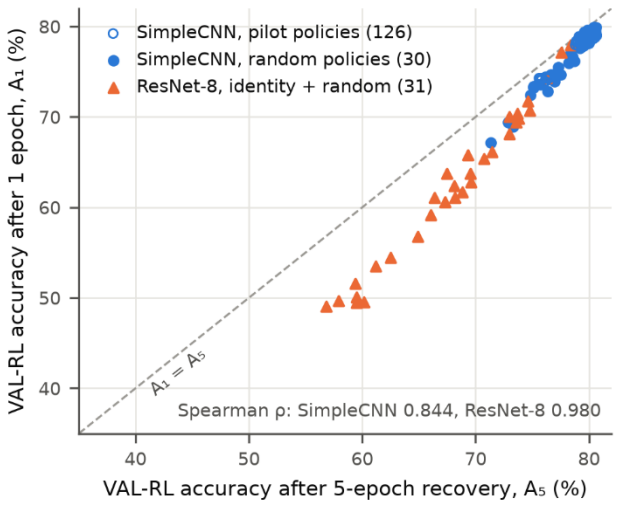


Fig. 2. VAL-RL accuracy after the 1-epoch proxy recovery against the full 5-epoch recovery for the validation policies of both networks. The dashed line marks equality.

D. Latency Room under an Accuracy Constraint

For ResNet-8, we computed the 1-epoch proxy for all 1,296 policies. The calibration sample of 48 policies, eight per bin of q_1, contained 14 policies with $q_5 \geq 0.98$ (Table IV). The rule gave $\tau = 0.960$, with recall 1.00 and precision 0.875 on the sample. In the full table, 52 policies satisfy $q_1 \geq \tau$, but the fastest of them is only 1.098× faster than dense, and the fastest truly feasible calibration policy is 1.089× faster. Speed-ups of 1.13–1.22× occur only among calibration policies whose 5-epoch retention is below 0.98 (Fig. 3). The pre-registered requirement of at least one feasible policy with a 1.10× speed-up is therefore not met, and the latency-aware RL search was not started.

TABLE IV. RESNET-8 LATENCY ROOM AT 98% RETAINED ACCURACY

Quantity	Value
Calibration policies	48
With $q_5 \geq 0.98$	14
Proxy threshold τ	0.960
Recall / precision at τ	1.00 / 0.875
Policies with $q_1 \geq \tau$ (of 1,296)	52
Latency of these policies	0.702–0.770 ms
Best speed-up (required ≥ 1.10×)	1.098×
Best truly feasible (calibration)	1.089×

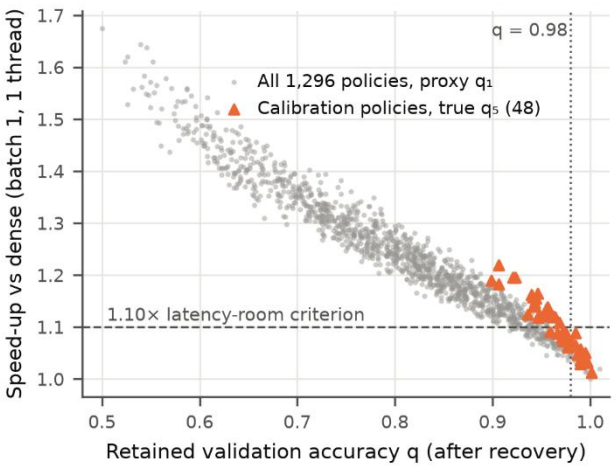


Fig. 3. ResNet-8 speed-up against retained accuracy: proxy q_1 for all 1,296 policies (grey) and 5-epoch q_5 for the 48 calibration policies (orange). No policy at $q \geq 0.98$ reaches the 1.10× line.

V. DISCUSSION

A. What the Reward Should Measure

On this CPU, a pruning reward based on parameter count would steer an agent toward the fully connected layer of SimpleCNN, which holds 85% of the parameters but 5% of the MACs. Such policies save memory but not time. MAC reduction ranks policies almost as measured latency does (ρ up to 0.98), so a MAC-based reward is a sound, hardware-independent first approximation. Measured latency adds the fixed overhead and shape-specific effects that MACs miss. Because the action space is finite, an exhaustive latency table costs little (the four tables took about 2.5 hours) and removes timing noise from the reward entirely.

B. Why ResNet-8 Leaves Little Room

ResNet-8 is already compact, with 78,042 parameters. About 43% of its dense latency is fixed overhead (0.33 of 0.77 ms), and its residual stages are fragile under channel removal. Under a 2% accuracy tolerance after five recovery epochs, only lightly pruned policies survive. Their MAC reductions, at most 18% among the proxy-feasible policies, translate into at most 9% less latency. The bottleneck is the accuracy constraint, not the search algorithm: an exhaustive search of the proxy table finds no proxy-feasible policy at 1.10×, although the proxy threshold kept every truly feasible calibration policy.

C. Implications for Hardware-Aware RL

The latency room of a pruning problem can be measured before any agent is trained, and it should be. Here, an exhaustive latency table, an accuracy proxy validated against the full recovery, and a feasibility check showed that a latency-aware agent would compete with simpler objectives over a band of less than 10%. Larger networks, a looser accuracy tolerance, longer recovery, or batched inference (where configuration C reached a 2.76× speed-up in calibration) are the settings in which hardware-aware RL has room to matter.

D. Limitations

All latencies come from one laptop CPU in one thread configuration under the default Windows power plan, and the absolute latency shifted by 16% between days, so only relative gains are interpreted. The networks are small and CIFAR-

scale. All accuracies are validation accuracies with a single recovery seed; the test set was deliberately not used. The 0.98 tolerance and the 1.10× threshold are conventions fixed in advance, and other values would change the conclusion quantitatively.

VI. Conclusion

Exhaustive measurement of 5,184 physically compacted pruning policies shows that parameter sparsity is a poor proxy for CPU latency, while MAC count is a good one. It also shows that structured pruning of a compact ResNet-8 leaves less than 10% of latency to gain once 98% of the recovered accuracy must be kept. Hardware-aware pruning should therefore reward measured or MAC-based cost, and the latency headroom of a pruning problem should be established before an RL search is built on it.

References

[1] S. Han, J. Pool, J. Tran, and W. J. Dally, "Learning both weights and connections for efficient neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 1135–1143.

[2] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[3] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[4] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 16091–16101.

[5] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "AMC: AutoML for model compression and acceleration on mobile devices," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 784–800.

[6] J. Lee, S. Park, S. Mo, S. Ahn, and J. Shin, "Layer-adaptive sparsity for the magnitude-based pruning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[7] U. Evci, T. Gale, J. Menick, P. S. Castro, and E. Elsen, "Rigging the lottery: Making all tickets winners," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2020, pp. 2943–2952.

[8] T.-J. Yang, A. Howard, B. Chen, X. Zhang, A. Go, M. Sandler, V. Sze, and H. Adam, "NetAdapt: Platform-aware neural network adaptation for mobile applications," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 285–300.

[9] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.

[10] B. Wu *et al.*, "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 10734–10742.

[11] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in *Proc. Mach. Learn. Syst. (MLSys)*, 2020.

[12] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," *J. Mach. Learn. Res.*, vol. 22, no. 241, pp. 1–124, 2021.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[14] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[15] A. Krizhevsky, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, ON, Canada, Tech. Rep., 2009.

[16] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 8024–8035.